DISJOINT SETS

Minimum Spanning Tree

Connected Components

Courtesy: Umama Rahman

Disjoint Sets

- Some applications require maintaining a collection of disjoint sets.

- A Disjoint Set ***S*** is a collection of sets $S_1,.....S_n$

  where $\forall_{i \neq j} S_i \cap S_j = \phi$

- Each set has a representative which is a member of the set (usually the minimum if the elements are comparable)

Disjoint Set Operations

- Make-Set(x) – Creates a new set S_x where x is it's only element (and therefore it is the representative of the set).
  $O(1)$ time.

- Union(x, y) – Replaces S_x, S_y by $S_x \cup S_y$.
  One of the elements of $S_x \cup S_y$ becomes the representative of the new set.
  $O(\log n)$ time.

- Find(x) – Returns the **representative** of the set containing x
  $O(\log n)$ time.

Disjoint set data structure

Consider 5 elements: 0, 1, 2, 3, 4

Make set for all : {0}, {1}, {2}, {3}, {4}

Union(0, 3) : {0, 3} , {1}, {2}, {4}

Union(1, 3) : {0, 1, 3} , {2}, {4}

Union(0, 1) : no need to union

Union(2, 4) : {0, 1, 3} , {2, 4}

Union(1, 4) : {0, 1, 3, 2, 4}

No of elements = n

No of unions required to combine all the elements into a single set = n - 1

Algorithm for Disjoint-Set Forest

```
MAKE-SET(x)
1. p[x]←x
```

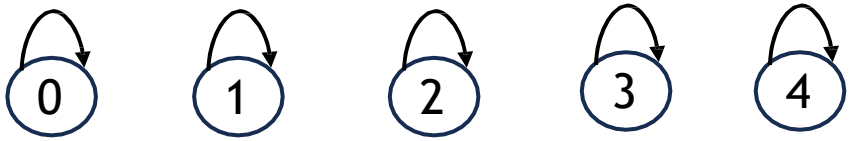
```
UNION(x, y)
1. a = FIND-SET(x)
2. b = FIND-SET(y)
3. p[a] = b
```

```
FIND-SET(x)
1. if x≠ p[x]
2.       return FIND-SET(p[x])
3. return p[x]
```

Disjoint set data structure representation

Consider 5 elements: 0, 1, 2, 3, 4

Make set for all :



	0	1	2	3	4
parent	0	1	2	3	4

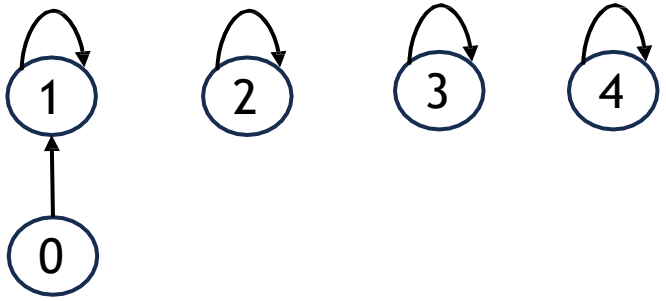
Union(0, 1) :

a = Find(0) = 0

b = Find(1) = 1

p[a]=b

p[0] = 1



	0	1	2	3	4
p	1	1	2	3	4

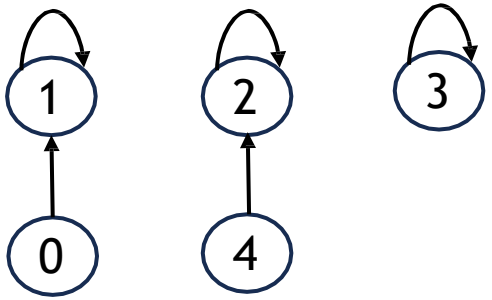
Union(4, 2) :

a = Find(4) = 4

b = Find(2) = 2

p[a]=b

p[4] = 2



	0	1	2	3	4
p	1	1	2	3	2

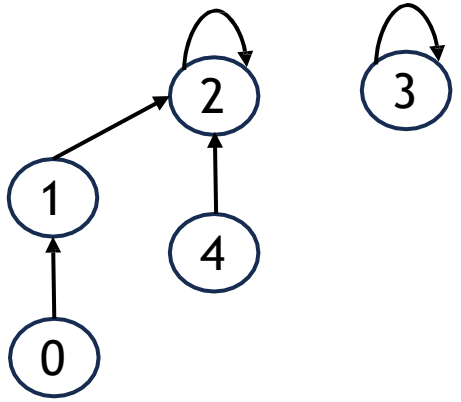
Union(0, 2) :

a = Find(0) = 1

b = Find(2) = 2

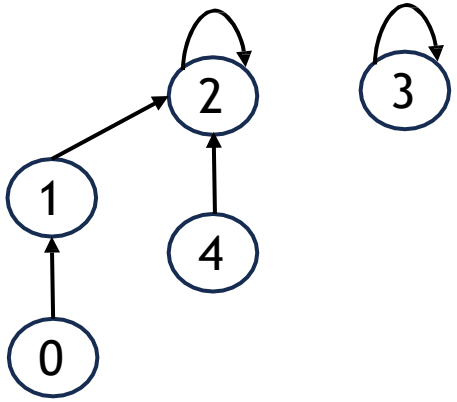
p[a]=b

p[1] = 2



	0	1	2	3	4
p	1	2	2	3	2

Disjoint set data structure representation



	0	1	2	3	4
p	1	2	2	3	2

Union(0, 4) :

a = Find(0) = 2

b = Find(4) = 2

Representative same implies

They both are in the same set

Or they are already connected

No, need to union

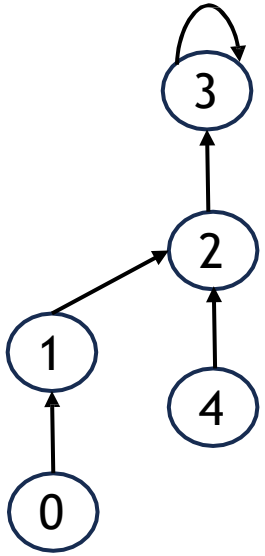
Union(1, 3) :

a = Find(1) = 2

b = Find(3) = 3

p[a]=b

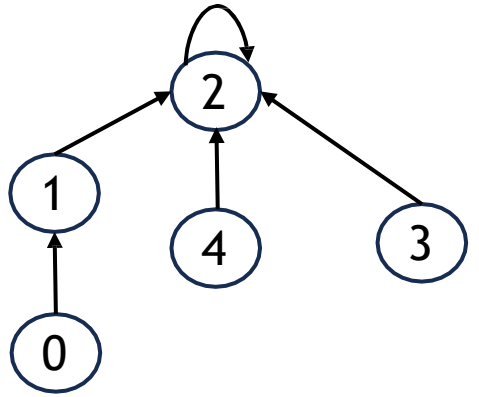
p[2] = 3



	0	1	2	3	4
p	1	2	3	3	2

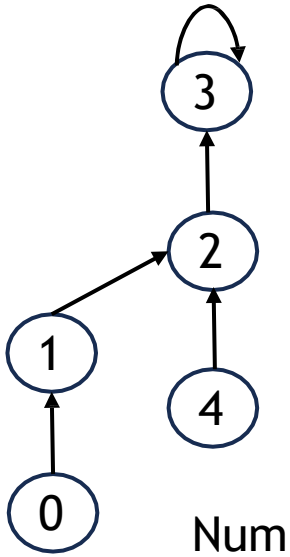
If we did

p[3] = 2



	0	1	2	3	4
p	1	2	2	2	2

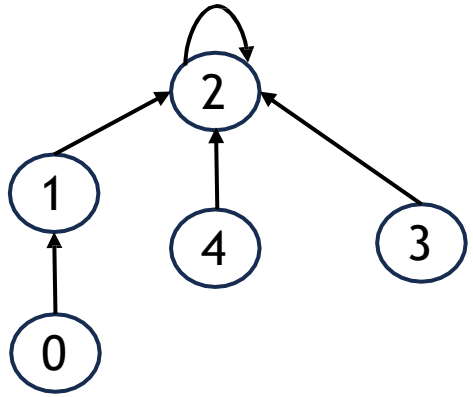
Time complexity find operation



Find(0) → Find(1) → Find(2) → Find(3)

3
3
3

Number of
recursive function calls = 4

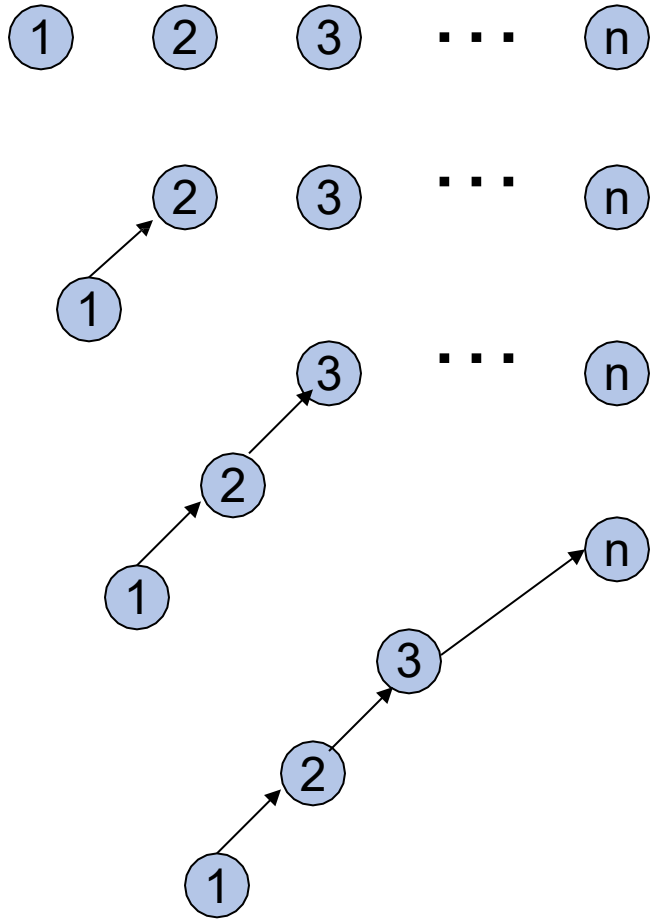


Find(0) → Find(1) → Find(2)

2
2

Number of
recursive function calls = 3

A Bad Case



Union(1,2)

Union(2,3)

:
:

Union(n-1, n)

Find(1) n steps!!

Improving Find

Can we do better? *Yes!*

Improve union so that *find* only takes $\Theta(\log n)$

- Union-by-size/rank

Improve find so that it becomes even better!

- Path compression

Union by Rank & Path Compression Heuristics

- Union by Rank: Each node is associated with a rank, which is the upper bound on the height of the node (i.e., the height of subtreerooted atthe node), then when UNION, let the root with smaller rank point to the root with larger rank.

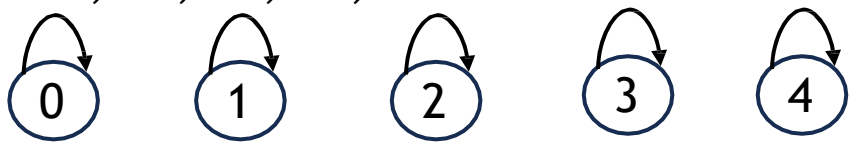
- Path Compression: used inFIND-SET(x) operation, make each node in thepath from x to the root directly point to the root. Thus reduce the treeheight.

Goal : To improve the time complexity of find operation

Union by rank *heuristic*: to keep the height of the tree small

Consider 5 elements: 0, 1, 2, 3, 4

Make set for all :



	0	1	2	3	4
parent	0	1	2	3	4
Rank	0	0	0	0	0

```
void Union(int a, int b){
        x = Find(a);
        y = Find(b);

        if(Rank[x] > Rank[y]){
                p[y] = x;
         }else{
                p[x] = y;
         }
}
```

Union by rank : to keep the height of the tree small

```
void Union(int a, int b){
        x = Find(a);
        y = Find(b);

        if(Rank[x] > Rank[y]){
                p[y] = x;
        }else{
                p[x] = y;
        }
}
```

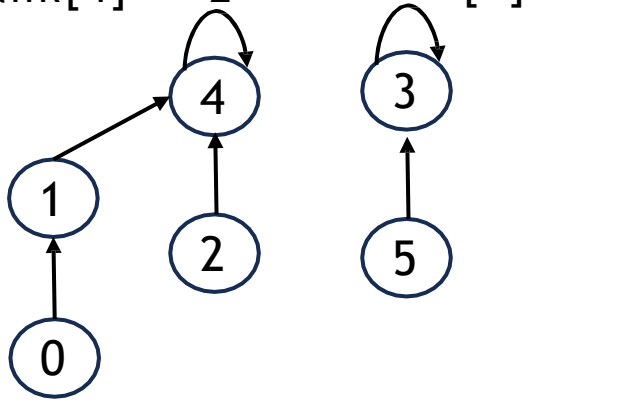
Union(0, 5)

x = Find(0) = 4
y = Find(5) = 3

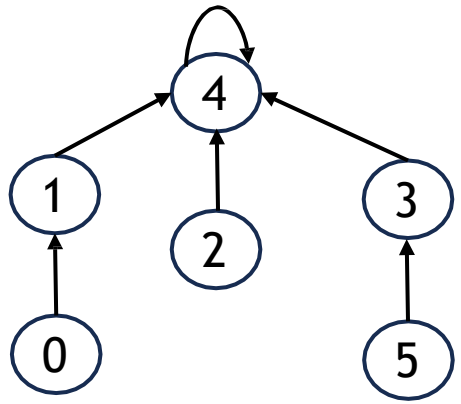
Rank[x] = Rank[4] = 2
Rank[y] = Rank[3] = 1

Rank[x] > Rank[y]
p[y] = x
p[3] = 4

Rank[4] = 2 Rank[3] = 1



Union(0, 5)



	0	1	2	3	4	5
parent	1	4	4	3	4	3
Rank	0	1	0	1	2	0

	0	1	2	3	4	5
parent	1	4	4	4	4	3
Rank	0	1	0	1	2	0

Union by rank : to keep the height of the tree small

```
void Union(int a, int b){
        x = Find(a);
        y = Find(b);

        if(Rank[x] > Rank[y]){
                p[y] = x;
        }else{
                p[x] = y;
        }
}
```

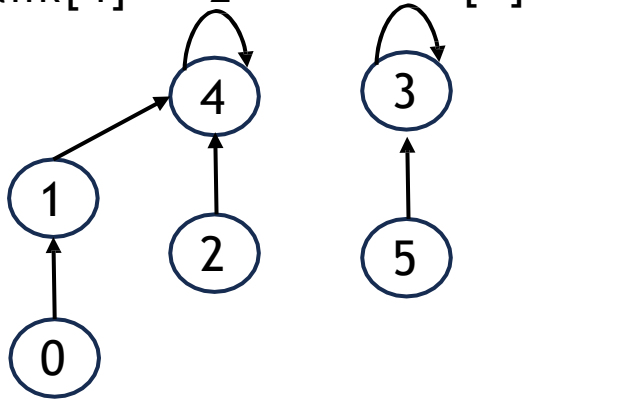
Union(3, 1)

x = Find(3) = 3
y = Find(1) = 4

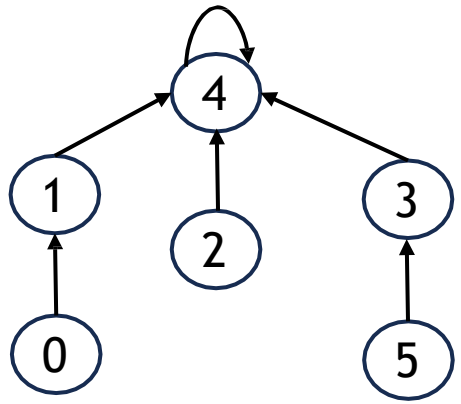
Rank[x] = Rank[3] = 1
Rank[y] = Rank[4] = 2

Rank[x] < Rank[y]
p[x] = y
p[3] = 4

Rank[4] = 2 Rank[3] = 1



Union(3, 1)



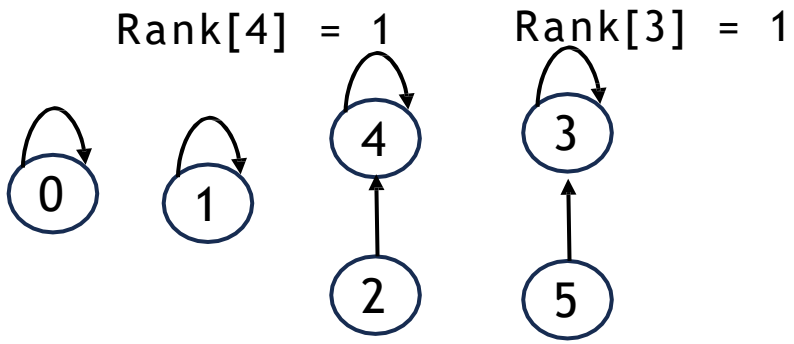
	0	1	2	3	4	5
parent	1	4	4	3	4	3
Rank	0	1	0	1	2	0

	0	1	2	3	4	5
parent	1	4	4	4	4	3
Rank	0	1	0	1	2	0

Union by rank : to keep the height of the tree small

```
void Union(int a, int b){
        x = Find(a);
        y = Find(b);

        if(Rank[x] > Rank[y]){
                p[y] = x;
        }else{
                p[x] = y;
        }
}
```



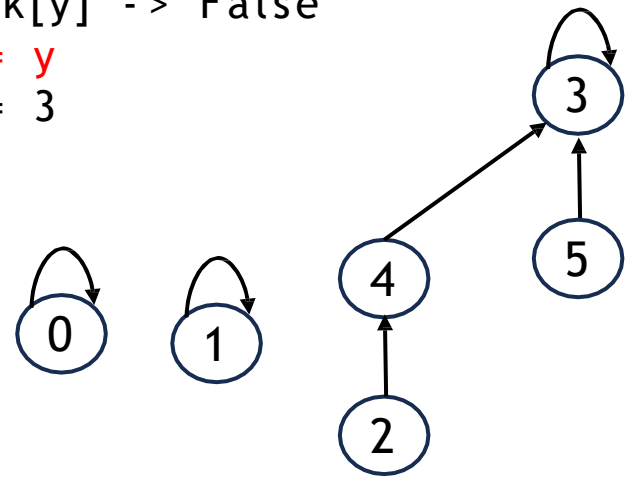
Union(2, 3)

Union(2, 3)

x = Find(2) = 4
y = Find(3) = 3

Rank[x] = Rank[4] = 1
Rank[y] = Rank[3] = 1

Rank[x] > Rank[y] -> False
p[x] = y
p[4] = 3



	0	1	2	3	4	5
parent	0	1	4	3	4	3
Rank	0	0	0	1	1	0

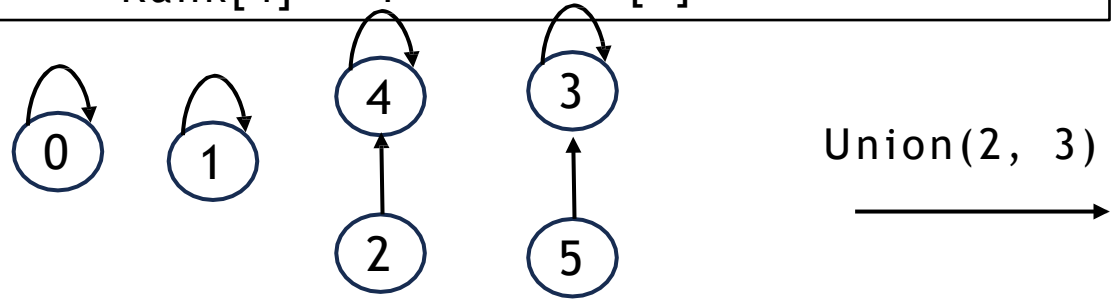
	0	1	2	3	4	5
parent	0	1	4	3	3	3
Rank	0	0	0	2	1	0

Union by rank : to keep the height of the tree small

```
void Union(int a, int b){
        x = Find(a);
        y = Find(b);

        if(Rank[x] > Rank[y]){
                p[y] = x;
        }else{
                p[x] = y;
                if (Rank[x] == Rank[y])
                   Rank[y]++;
        }
}
```

Rank[4] = 1 Rank[3] = 1



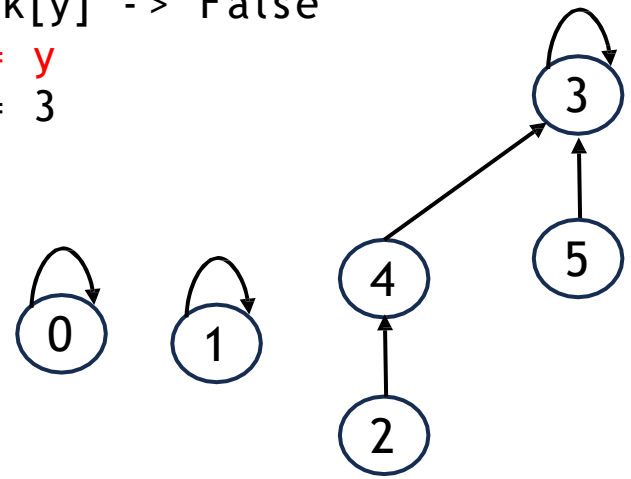
Union(2, 3)

Union(2, 3)

x = Find(2) = 4
y = Find(3) = 3

Rank[x] = Rank[4] = 1
Rank[y] = Rank[3] = 1

Rank[x] > Rank[y] -> False
p[x] = y
p[4] = 3

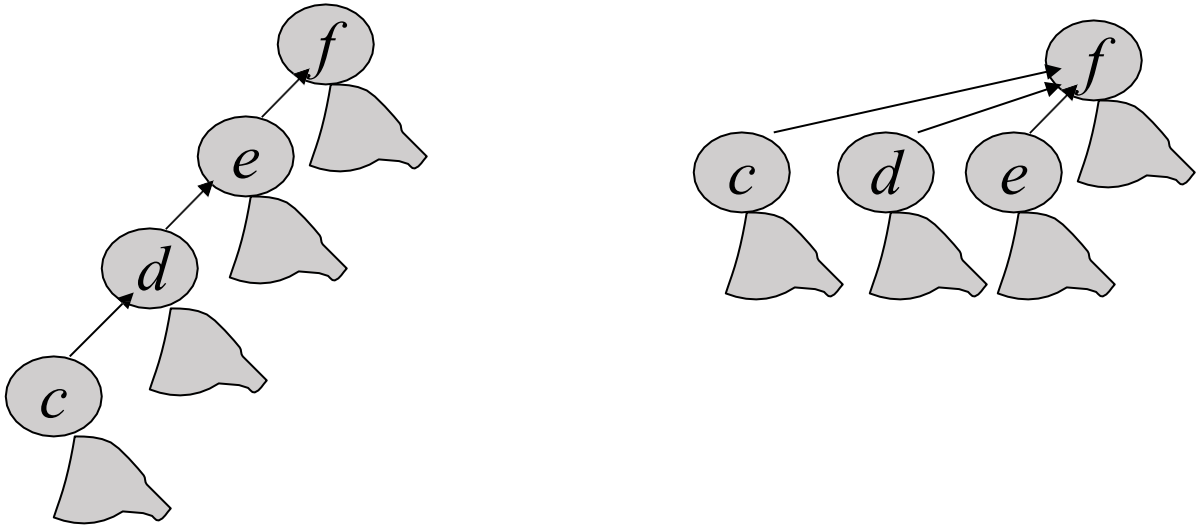


	0	1	2	3	4	5
parent	0	1	4	3	4	3
Rank	0	0	0	1	1	0

	0	1	2	3	4	5
parent	0	1	4	3	3	3
Rank	0	0	0	2	1	0

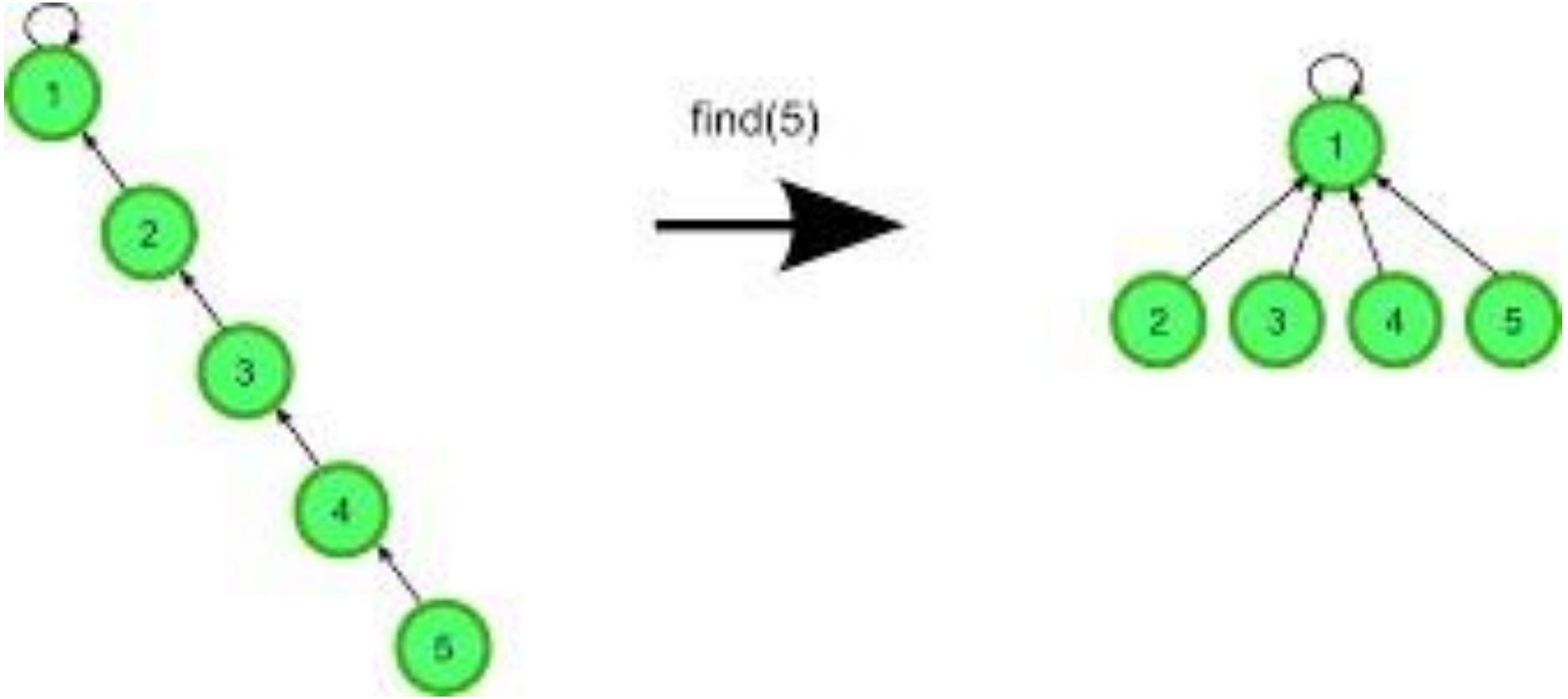
Path compression : to reduce the time complexity of Find

Path Compression: used in FIND(x) operation, make **each node in the path from x to the root directly point to the root**. Thus reduce the tree height. Speed up the `**Find**` operation by **flattening** the tree structure.

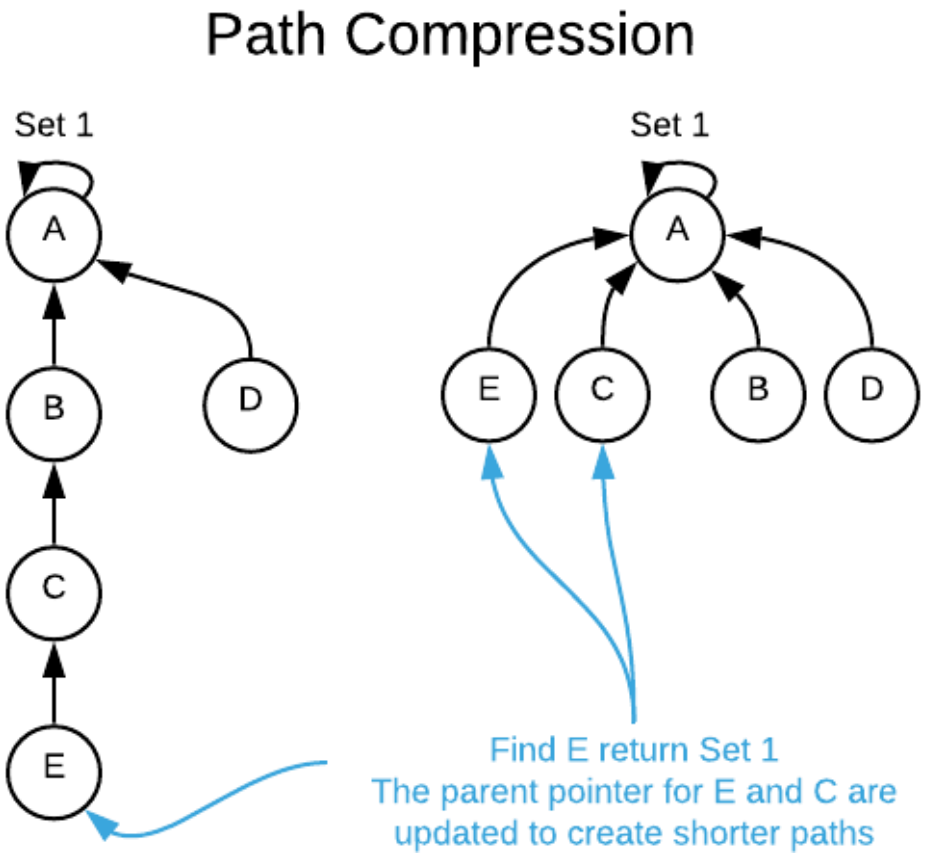


Path compression can cause a very deep tree to become very shallow

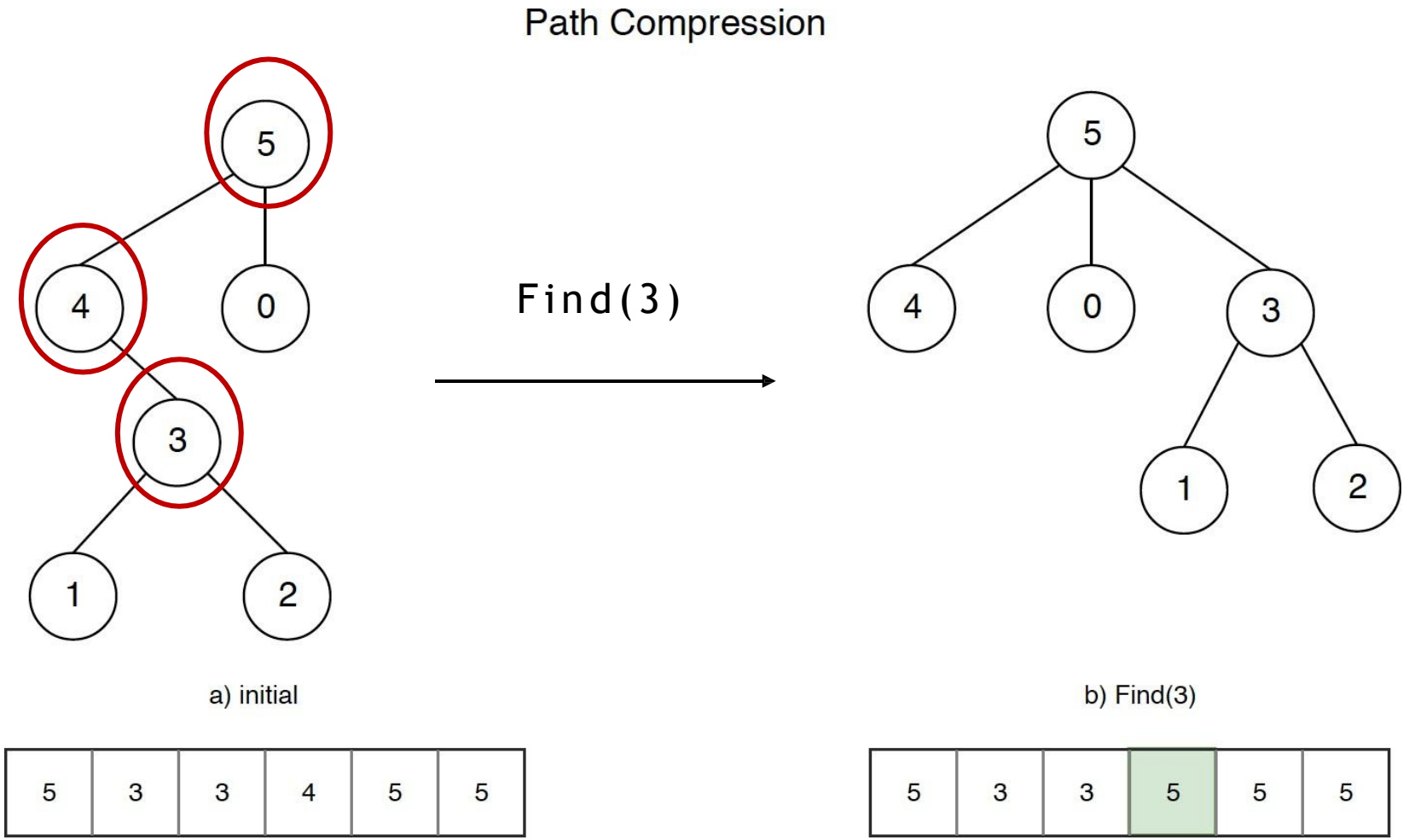
Path compression : to reduce the time complexity of Find



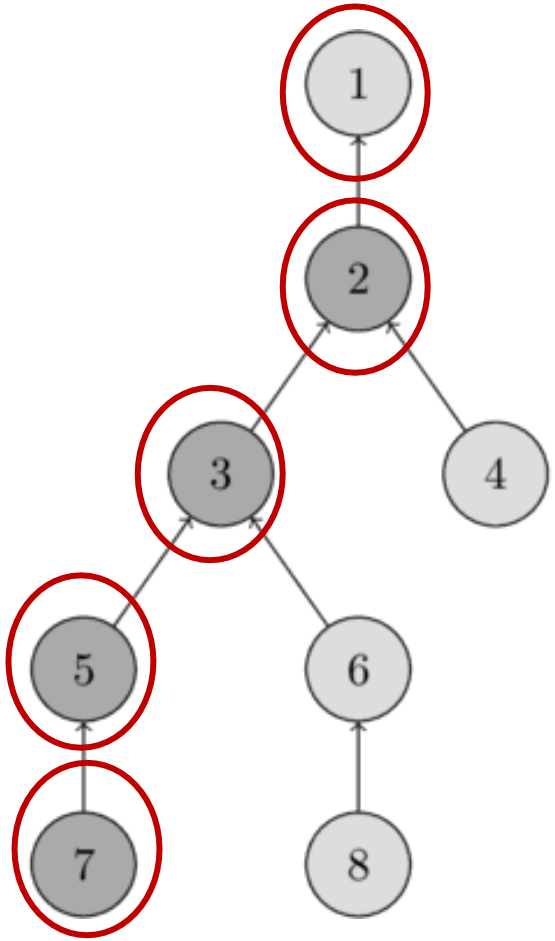
Path compression : to reduce the time complexity of Find



Path compression : to reduce the time complexity of Find

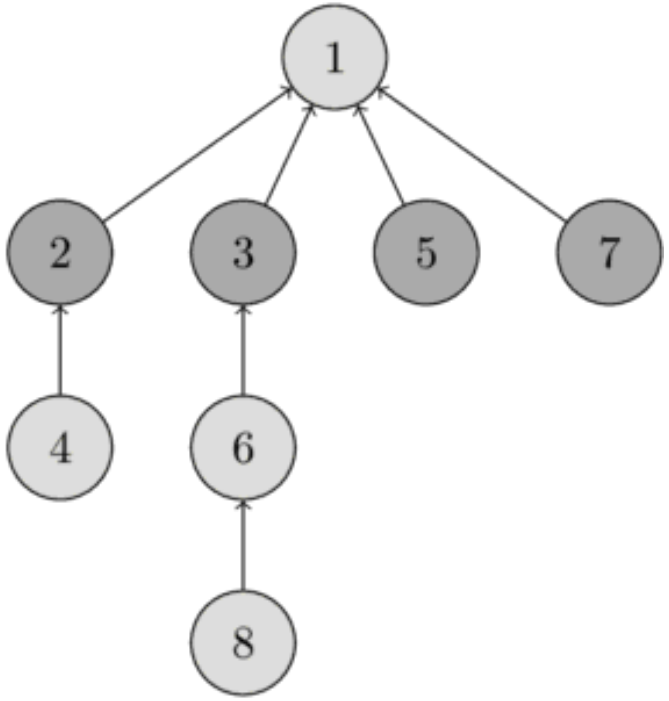


Path compression : to reduce the time complexity of Find



Find(7)

$\Longrightarrow$



Algorithm for Disjoint-Set Forest

With union by Rank and Path Compression Heuristic

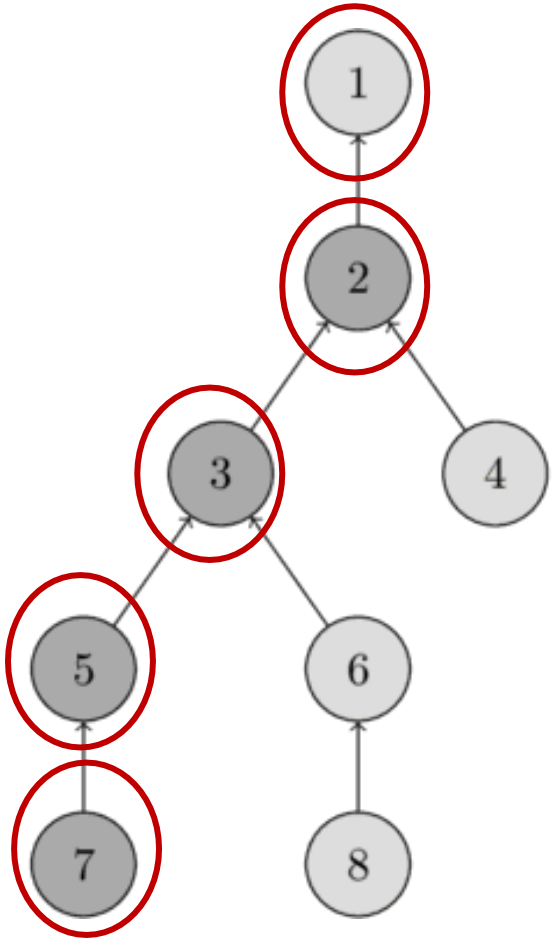
MAKE-SET(*x*)
1. $p[x] \leftarrow x$
2. $Rank[x] \leftarrow 0$

```
void UNION(int a, int b){
    x = FIND-SET(a);
    y = FIND-SET(b);

    if(Rank[x] > Rank[y]){
        p[y] = x;
    }else{
        p[x] = y;
        if (Rank[x] == Rank[y])
          Rank[y]++;
    }
}
```

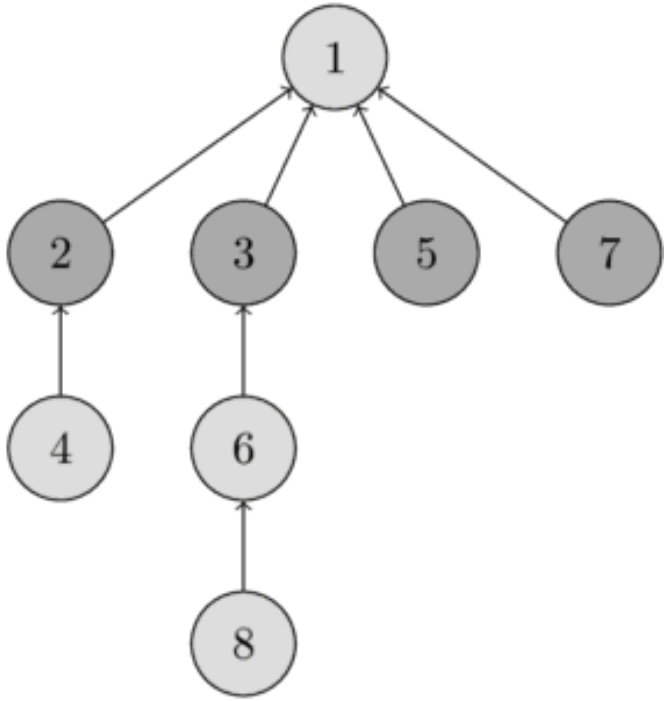
FIND-SET(*x*)
1. **if** $x \neq p[x]$
2. **then** $p[x] \leftarrow$ **FIND-SET(**$p[x]$**)**
3. **return** $p[x]$

Path compression : to reduce the time complexity of Find



Find(7)

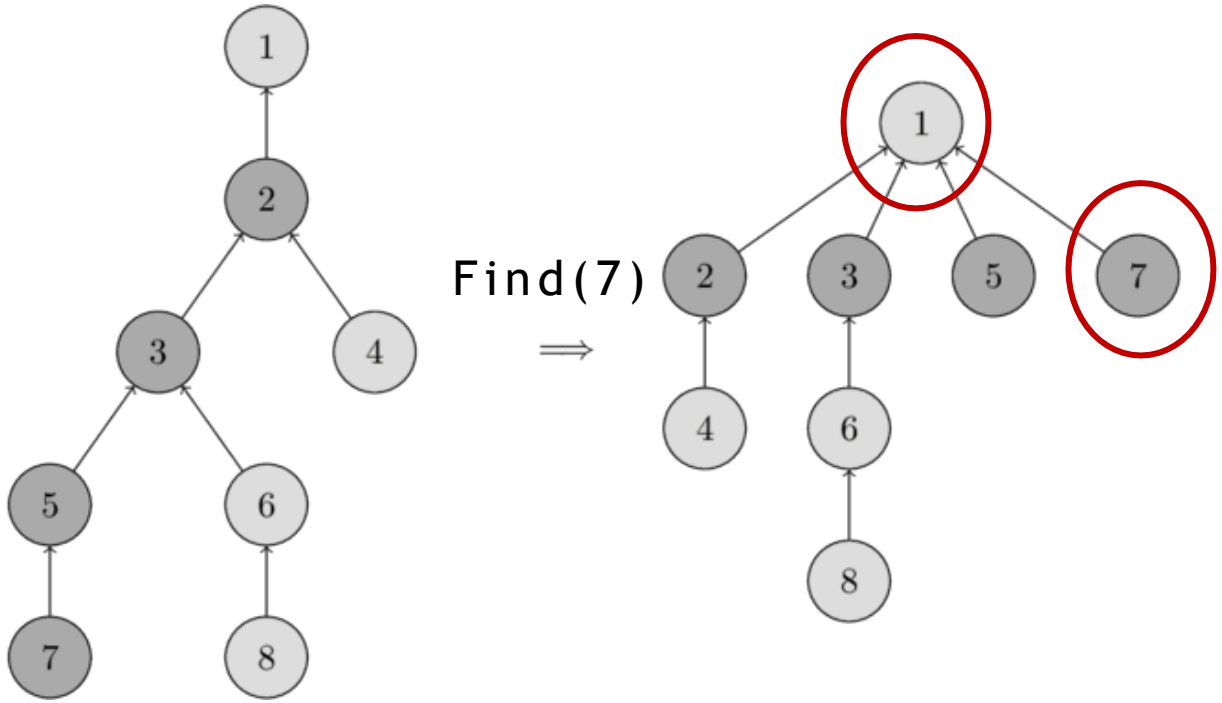
$\Longrightarrow$



p[7] = Find(7)
↓
p[5] = Find(5)
↓
p[3] = Find(3)
↓
p[2] = Find(2)
↓
Find(1)

1
1
1
1

Path compression : to reduce the time complexity of Find



p[7] = Find(7)

↓

Find(1)

1

This takes almost constant time

Effect : Overtime, as more `Find` operations are performed, the tree becomes flatter, and future `Find` operations will take constant timeon average.

QUESTION

- MakeSet(0)
- MakeSet(1)
- MakeSet(2)
- MakeSet(3)
- MakeSet(4)
- MakeSet(5)
- MakeSet(6)
- MakeSet(7)
- print(FIND-SET(7))
- Draw

- UNION(0, 4)
- UNION(6, 7)
- UNION(7, 2)
- print(FIND-SET(7))
- UNION(5, 7)
- UNION(6, 1)
- UNION(4, 3)
- print(FIND-SET(6))
- Draw

Exercise

The following table shows the parent array of a Disjoint set (Rooted tree implementation). Perform the followingoperations sequentially using **path compression and union-by-rank heuristic:**

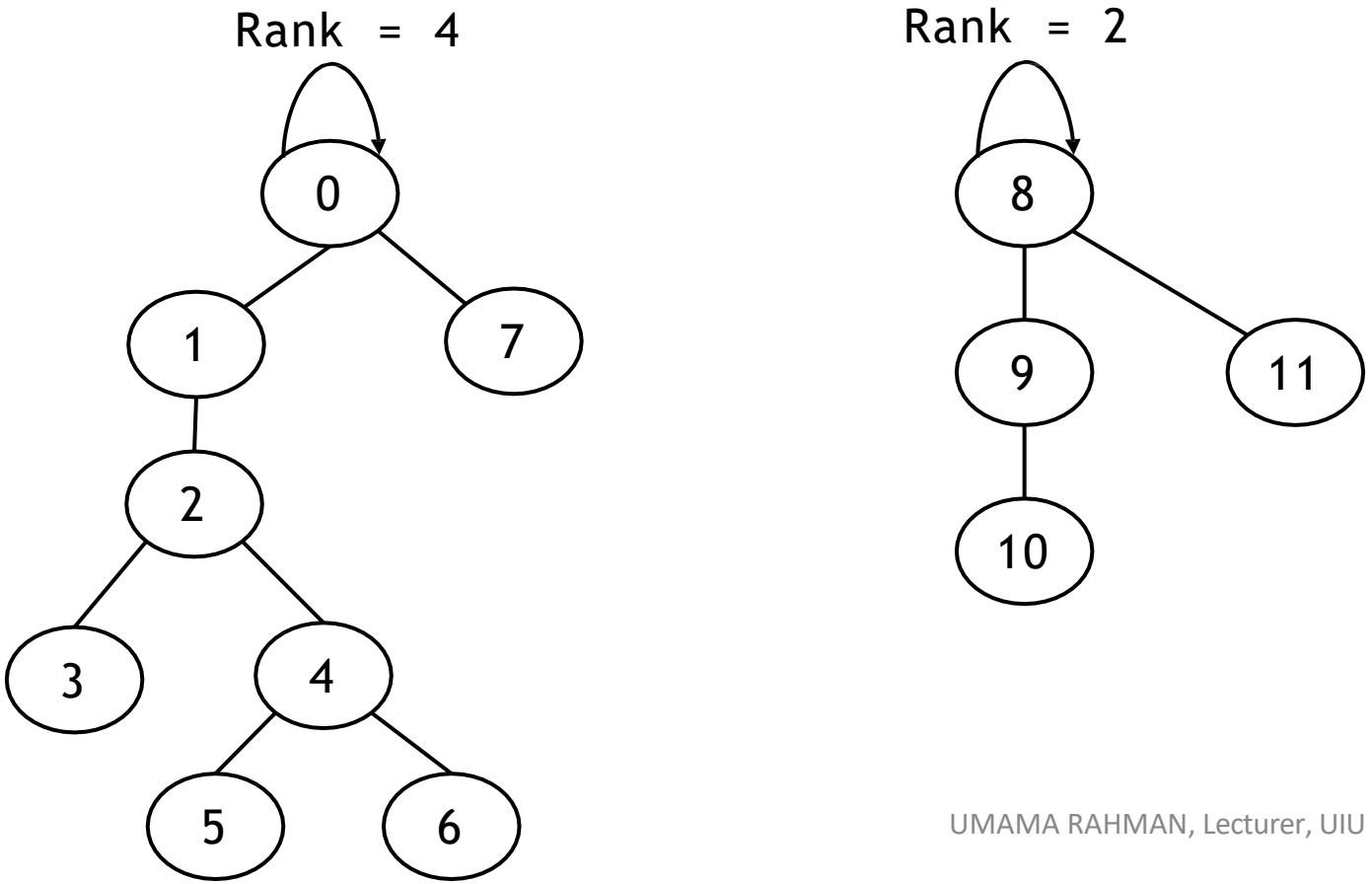
i. Draw the disjoint set forest
ii. Redraw the forest after Union(3,7)
iii. Redraw the forest after Union(5,10)
iv. What will be returned by Find-Set(9)?

Index	0	1	2	3	4	5	6	7	8	9	10	11
Parent	0	0	1	2	2	4	4	0	8	8	9	8

Solution

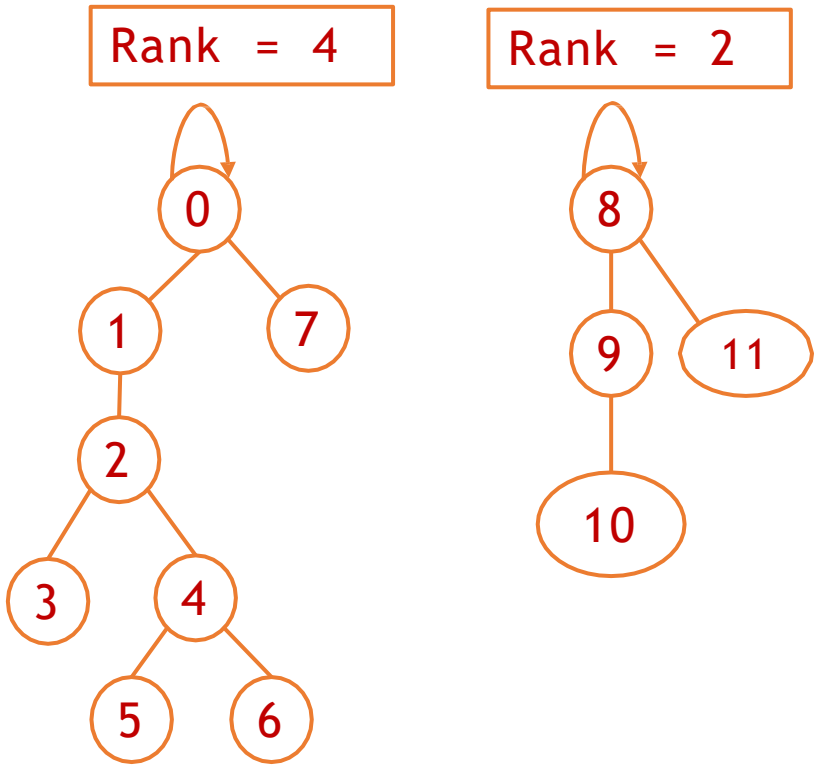
i. Draw the disjoint set forest

Index	0	1	2	3	4	5	6	7	8	9	10	11
Parent	0	0	1	2	2	4	4	0	8	8	9	8

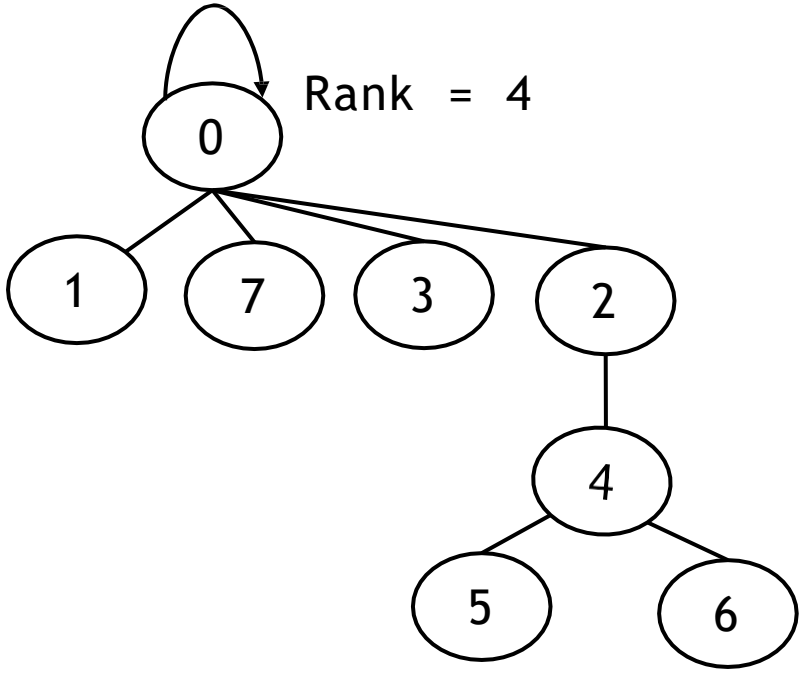


Solution

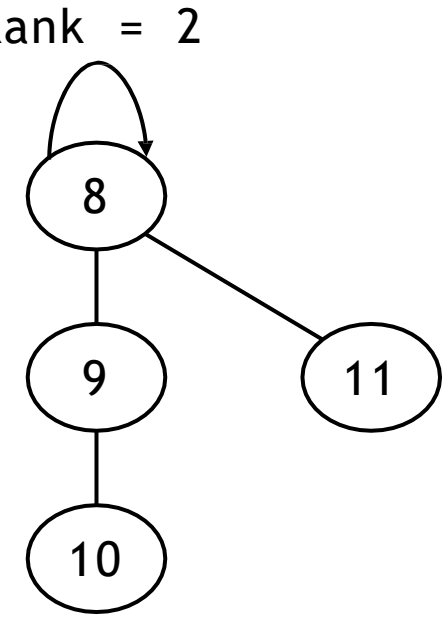
ii. Redraw the forest after Union(3,7)



Find(3) = 0



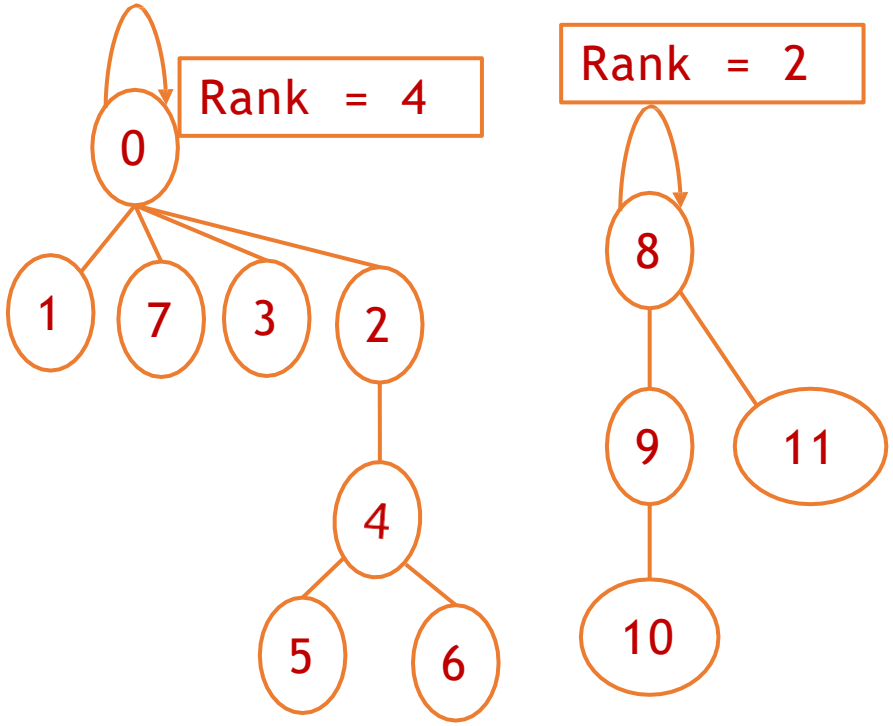
Find(7) = 0



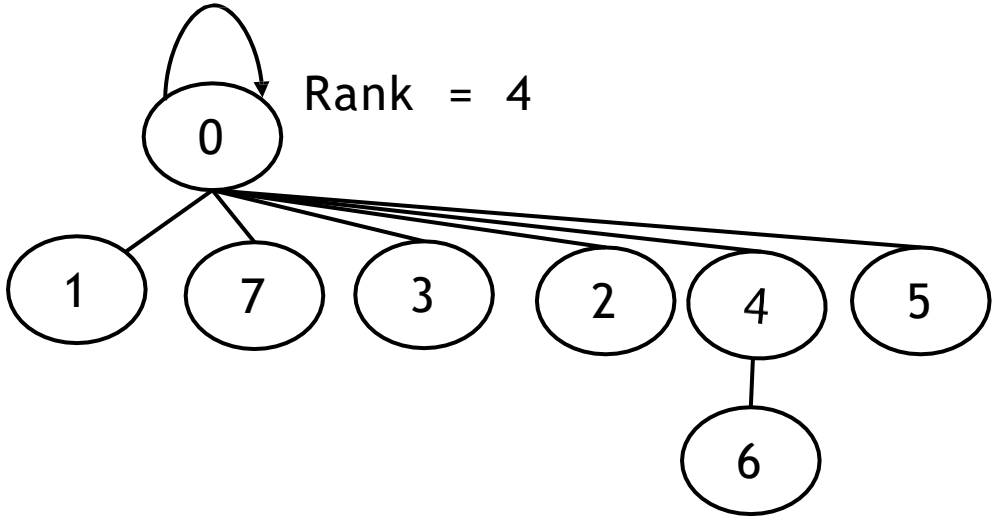
Since, Find(3) = Find(7)
They are in thesame set.
No need to union.

Solution

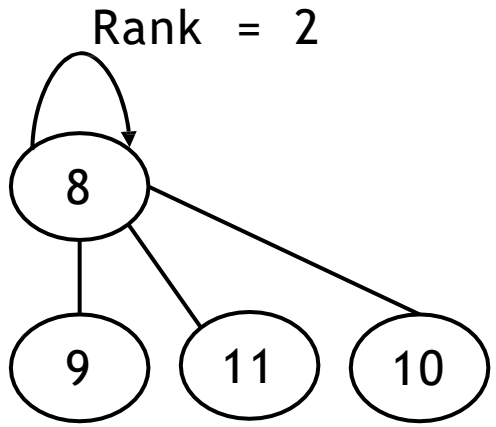
iii. Redraw the forest after Union(5,10)



Find(5) = 0

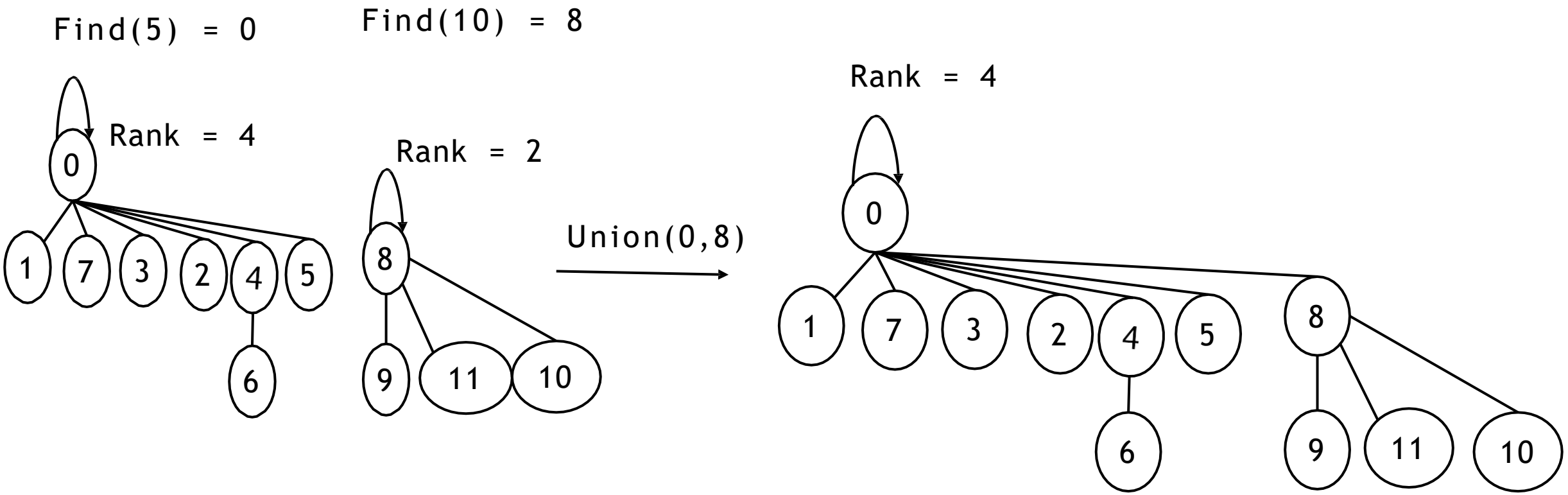


Find(10) = 8



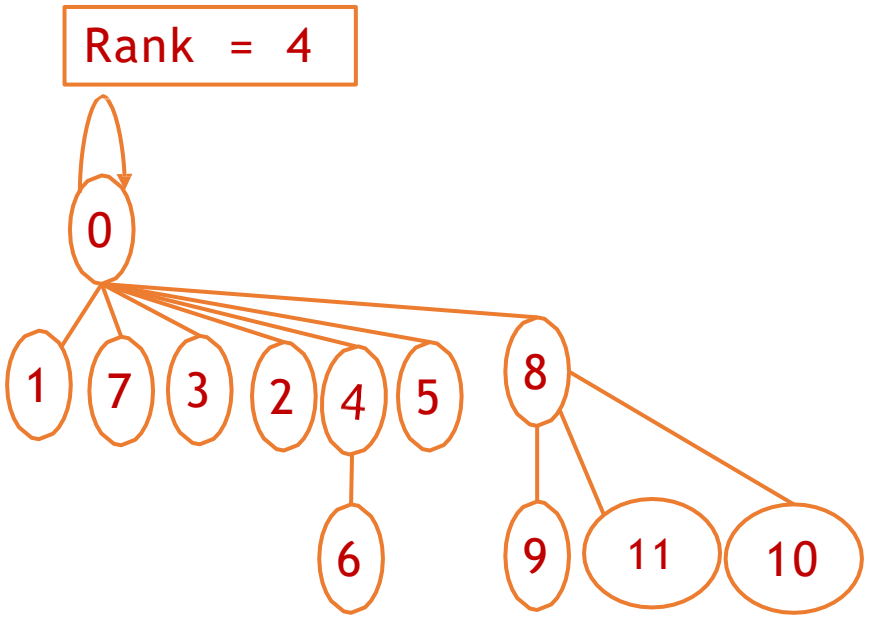
Solution

iii. Redraw the forest after Union(5,10)



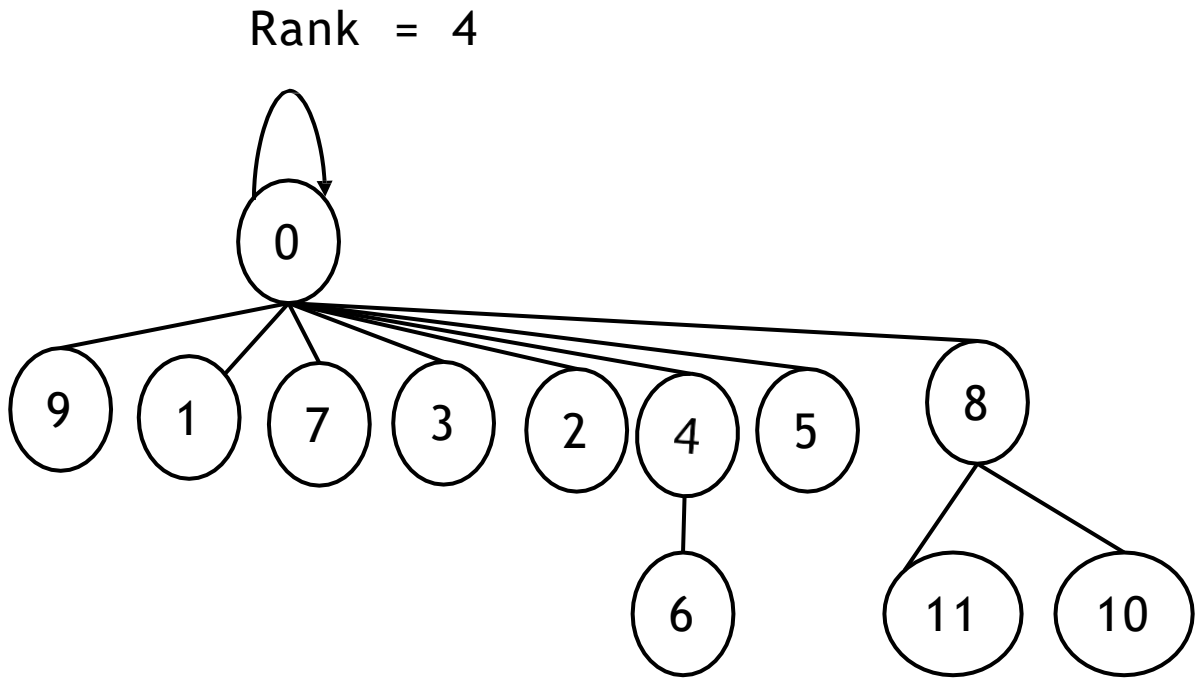
Solution

iv. What will be returned by Find-Set(9)?



Find(9)

Find(9) = 0



Time complexity of each operation

make_set	O(1)
Find_set	O(logn)
Union	O(logn)

Algorithm for Disjoint-Set Forest

With union by Rank and Path Compression Heuristic

MAKE-SET(*x*)
1. $p[x] \leftarrow x$
2. $Rank[x] \leftarrow 0$

```
void UNION(int a, int b){
    x = FIND-SET(a);
    y = FIND-SET(b);

    if(Rank[x] > Rank[y]){
        p[y] = x;
    }else{
        p[x] = y;
        if (Rank[x] == Rank[y])
          Rank[y]++;
    }
}
```

FIND-SET(*x*)
1. **if** $x \neq p[x]$
2. **then** $p[x] \leftarrow$ **FIND-SET(**$p[x]$**)**
3. **return** $p[x]$

Time complexity Analysis

- Consider asequenceofm makeSet, findSetand Union operations. what will be the total time complexity? [Totalnumberof elements is n]. Among them p aremake_set operations.

Solution:

Worst case running time for m MAKE-SET, UNION, FIND-SET operations is: $O(m \cdot \alpha(n))$, where $\alpha(n)$ is inverse Ackermann function and $\alpha(n) \leq 4$. So nearly linear in m. Then, total time complexity: O(m)

The find operation does not change: $O(\log n)$

Exercise

- Describe a data structure that supports the following operations:
  - find(x) – returns the representative of x
  - union(x, y) – unifies the groups of x and y
  - min(x) – returns the minimal element in the group of x

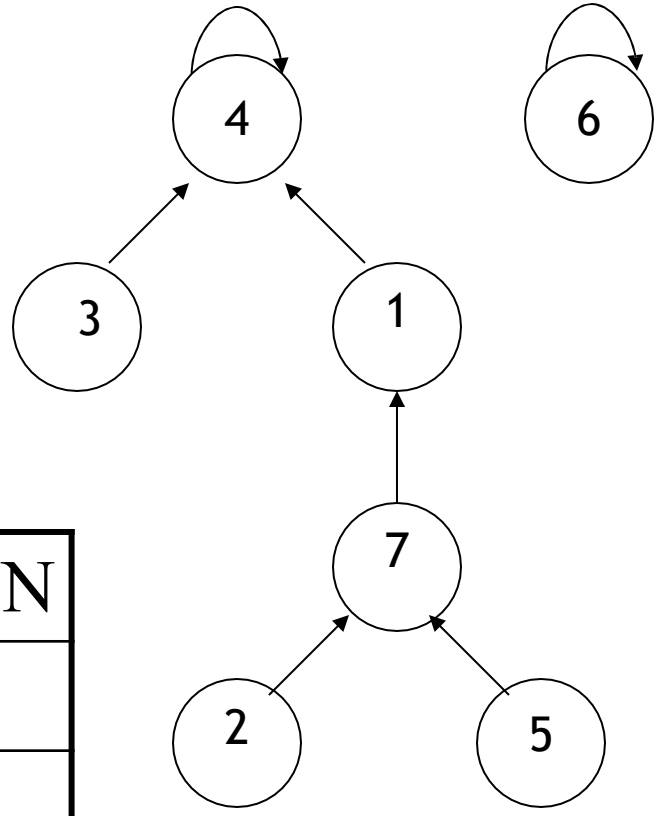
Solution

- We modify the disjoint set data structure so that we keep a reference to the minimal element in the group representative.

- The find operation does not change (log(n))

- The union operationis similar to the original union operation,and the minimal elementis the smallest between the minimalof the two groups

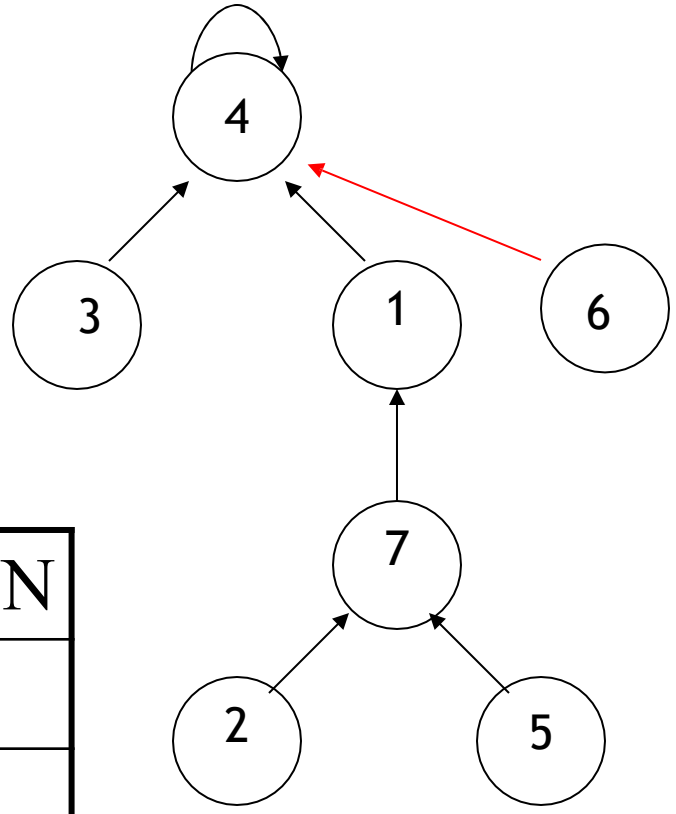
Example



	1	2	3	4	5	6	..	N
Parent	4	7	4	4	7	6		
min				1		6		

Example

- Executing union(4,6)



	1	2	3	4	5	6	..	N
Parent	4	7	4	4	7	4		
min				1		1		

Connected Components

Connected Components

- Given a graph G we first preprocess G to maintain a set of connectedcomponents

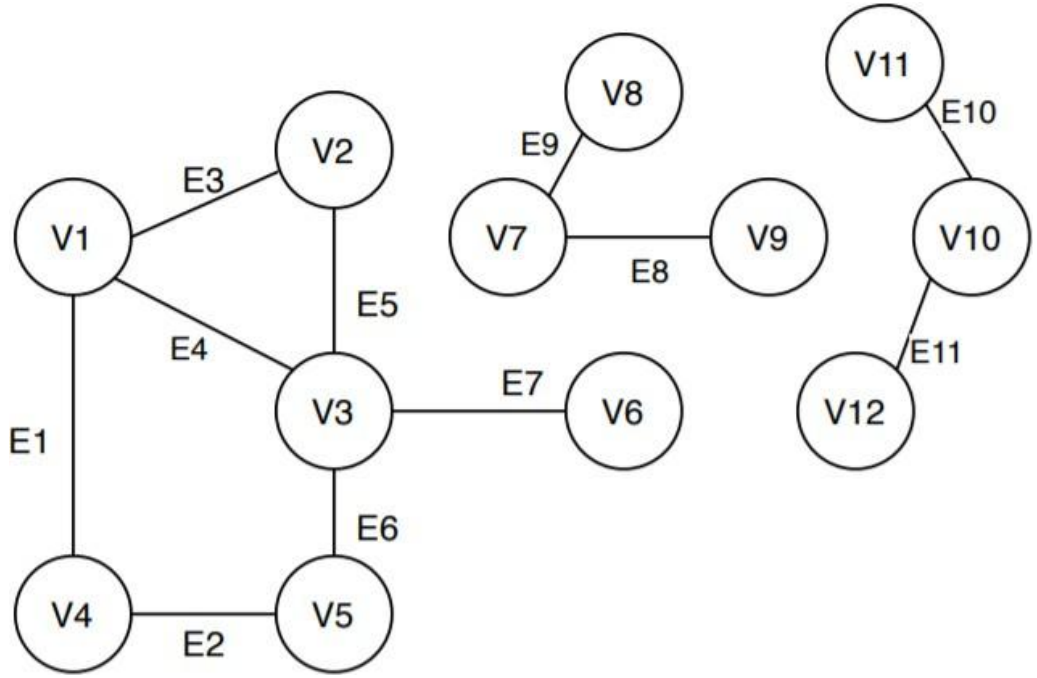
  CONNECTED_COMPONENTS(G)

- Later a series of queries can be executed to check if two vertexes are part of the same connected component

  SAME_COMPONENT(u, v)

Connected Components

A connected component of an undirected graph is a subgraph in which each pair of nodes is connected with each other via a path.In other words, a set of nodes forms a connectedcomponent in an undirected graph if any node from the set of nodes can reach any other node by traversing edges.

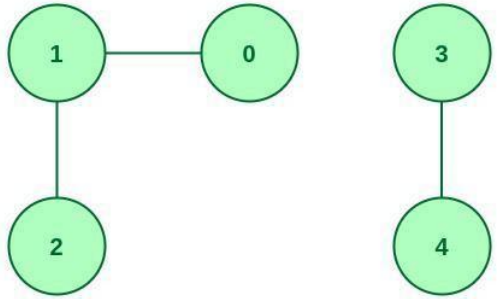


Connected Components

- The connected component set is always non-empty.
- If there is more than one connected component for a given graph, then the union of connected components will give the set of all vertices of the given graph
- Connected component sets are pairwise disjoint. That means if we take the intersection between two different connected component sets then the intersection is an empty set or a null set.

```
CONNECTED_COMPONENTS(G)
  for each vertex v in V[G] do
       MAKE_SET (v)

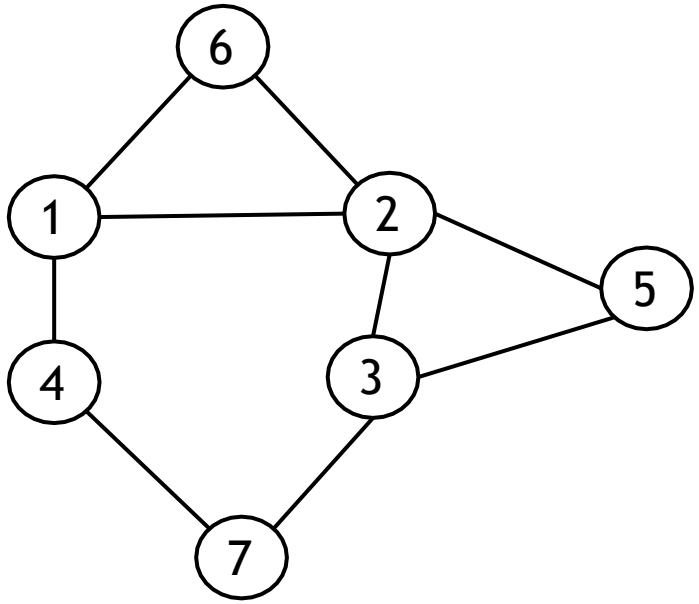
  for each edge (u, v) in E[G] do
       if FIND_SET(u) != FIND_SET(v) then
            UNION(u, v)
```

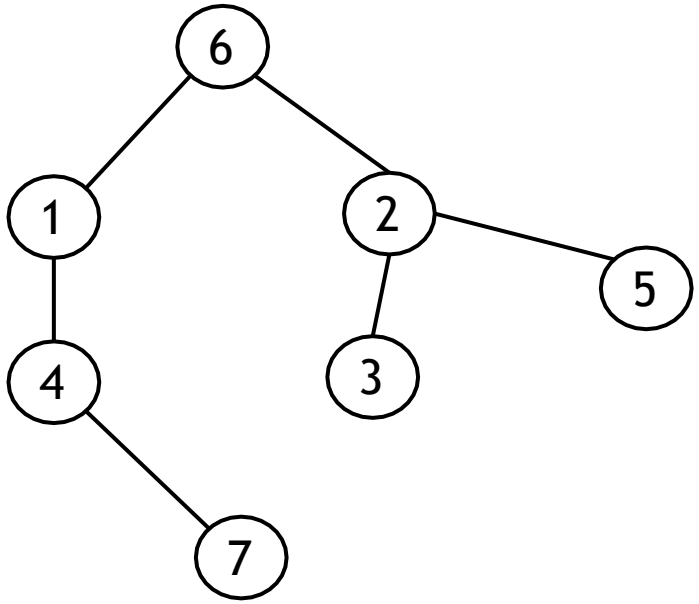



```
SAME_COMPONENT(u, v)
  if FIND_SET(u) == FIND_SET(v) then
       return TRUE
  else return FALSE
```

Connected Components in a graph

How many ***Union***s are required to connect the n nodes into k connected components using Disjoint set data structure?

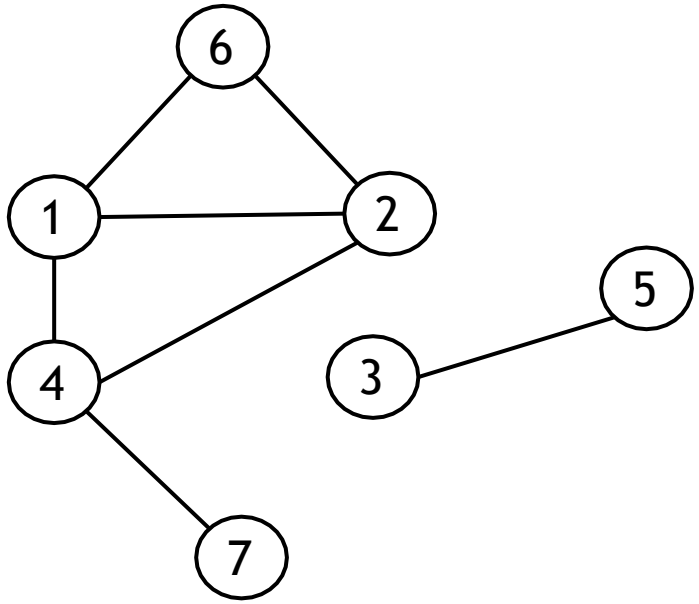


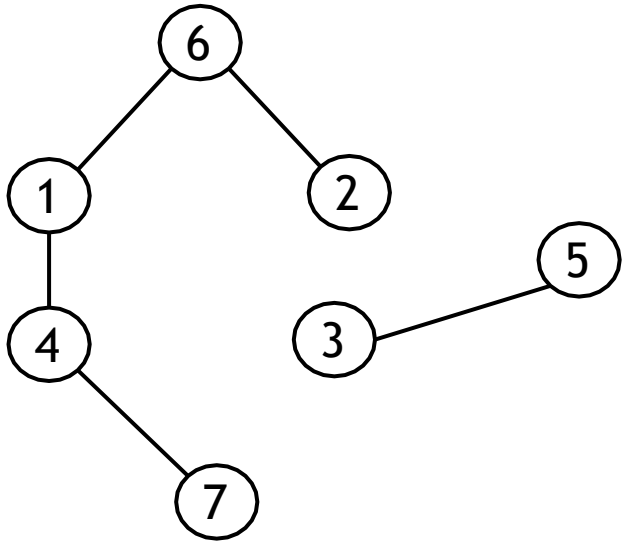


Number of vertices in the graph, n = 7
Number of connected components in the graph, k = 1

Connected Components in a graph

How many ***Union***s are required to connect the n nodes into k connected components using Disjoint set data structure?

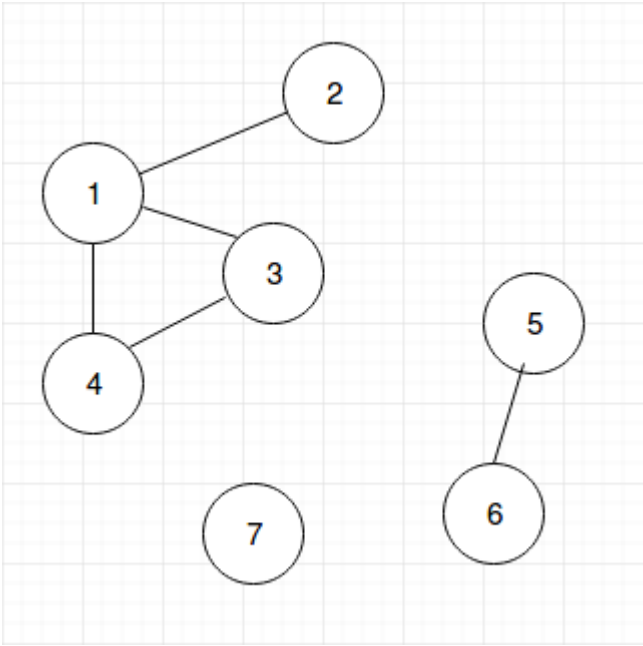


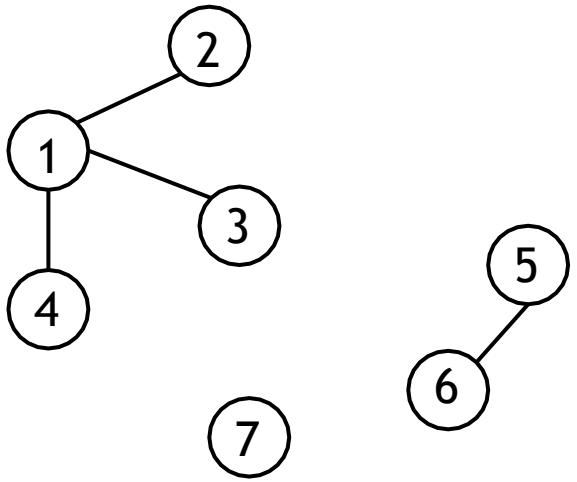


Number of vertices in the graph, n = 7
Number of connected components in the graph, k = 2

Connected Components in a graph

How many ***Union***s are required to connect the n nodes into k connected components using Disjoint set data structure?





Number of vertices in the graph, n = 7
Number of connected components in the graph, k = 3

Minimum number of Union operations required
to connect the nodes into k connected components = 4 = 7 – 3 = n - k

Connected Components in a graph

How many *Union*s are required to connect the n nodes into k connected components using Disjoint set data structure?

Number of vertices in the graph = n
Number of connected components in the graph = k

Minimum number of Union operations required
to connect the nodes into k connected components = n - k

```
CONNECTED_COMPONENTS(G)
  for each vertex v in V[G] do
        MAKE_SET (v)

  for each edge (u, v) in E[G] do
        if FIND_SET(u) != FIND_SET(v) then
              UNION(u, v)
```

How many times **make_set**, **find_set** and **union** will be called in this code ? There are n vertices, m edges and k connected components in the graph.

Make_set	n
Find_set	2m
Union	n-k

Minimum Spanning Tree (MST)

Kruskal's Algorithm

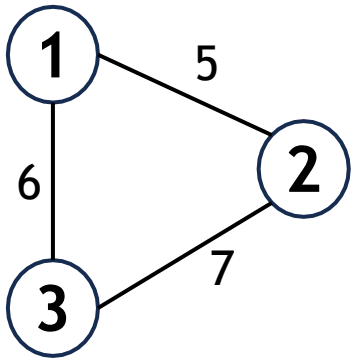
Kruskal Algorithm

```
Kruskal()
{
    T = ∅;
    for each v ∈ V
        MakeSet(v);

    sort E into nondecreasing order by weight w

    for each (u,v) ∈ E (in sorted order)
        if FindSet(u) ≠ FindSet(v)
            T = T U {{u,v}};
            Union(u,v);
}
```

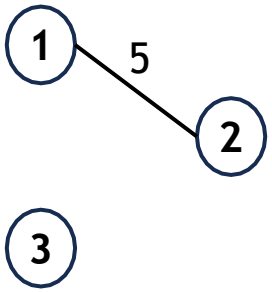
Minimum Spanning Tree (MST)



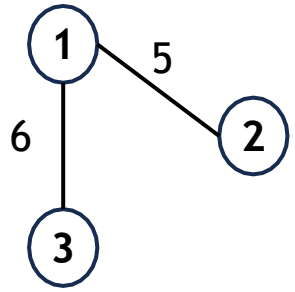
Solution: Sort E in ascending order of weight

1. (1, 2, 5)
2. (1, 3, 6)
3. (2, 3, 7)

Step-1: consider edge (1, 2, 5)
Find(1) ≠ Find(2)
According to Kruskal, take the edge and Join the trees containing 1 and 2 through union(1, 2)



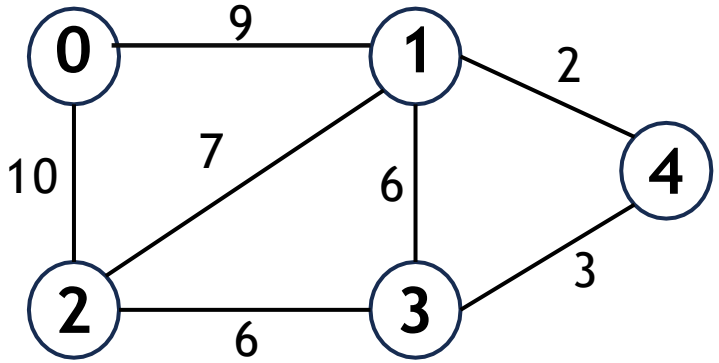
Step-2: consider edge (1, 3, 6)
Find(1) ≠ Find(3)
According to Kruskal, take the edge and Join the trees containing 1 and 3 through union(1, 3)

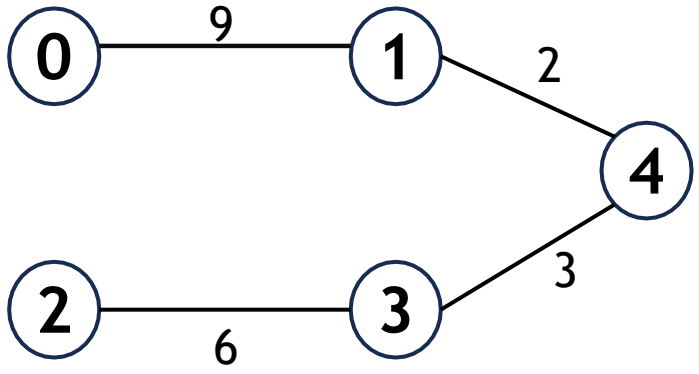


Step-3: consider edge (2, 3, 7)
Find(2) = Find(3)
Therefore, 2 and 3 are already in the same tree/set which means 2 and 3 are already connected.
Now, taking the edge (2, 3) will create a cycle. According to Kruskal, it doesn't take the edge.

For an edge (u, v, w) → if **Find**(u) == **Find**(v), they are already connected.
For an edge (u, v, w) → if **Find**(u) ≠ **Find**(v), they are already not connected. We take the edge (u, v) and connect them via **Union**(u, v)

Find Minimum Spanning Tree (MST) using Kruskal





Solution: Sort E in ascending order of weight

1. (1, 4, 2) √
2. (3, 4, 3) √
3. (1, 3, 6) ×
4. (2, 3, 6) √
5. (1, 2, 7) ×
6. (0, 1, 9) √
7. (0, 2, 10) ×

Time complexity of Kruskal Algorithm

```
Kruskal()
{
   T = ∅;
   for each v ∈ V
      MakeSet(v);

   sort E into nondecreasing order by weight w

   for each (u,v) ∈ E (in sorted order)
        if FindSet(u) ≠ FindSet(v)
             T = T U {{u,v}};
             Union(u,v);
}
```

No of vertices = V
No of edges = E

No of MakeSet, FindSet and Union required:

make_set	V
Find_set	2E
Union	V-1

Total operations = O(V+E)
Worst case time complexity for O(V+E) make_set and find_set and Union operation = O(V+E)

Time complexity for sorting edges = O(ElogE)

Total Time complexity: O(V+E) + O(ElogE) = O(ElogE)